Refatoração
Clean Code
Teste Unitário

Instituto Federal de Mato Grosso (IFMT)
Pontes e Lacerda

Conteúdo da aula

- **Introdução**

- Arranjos
- Matrizes
- Classe Array
- Classe ArrayList

- Conclusão
- Exercícios Propostos
- Referências bibliográficas

Conteúdo da aula

- Introdução

- **Arranjos**
- **Matrizes**
- **Classe Array**
- **Classe ArrayList**

- Conclusão
- Exercícios Propostos
- Referências bibliográficas

Conteúdo da aula

- Introdução

- Arranjos
- Matrizes
- Classe Array
- Classe ArrayList

- **Conclusão**
- **Exercícios Propostos**
- **Referências bibliográficas**

Introdução

-

Conteúdo da aula

- Introdução

- **Arranjos**
- Matrizes
- Classe Array
- Classe ArrayList

- Conclusão
- Exercícios Propostos
- Referências bibliográficas

Refatoração

- ***Definição***: Processo de mudar a estrutura interna de um programa, sem alterar a sua estrutura externa, com a finalidade de otimizar e/ou corrigir o programa

  - O programa realiza as mesmas funções
  - Código interno alterado
  - Objetivo é melhorar legibilidade de diminuir possíveis erros futuros

Refatoração

- ***Exemplo de "sujeiras" no código***:

  - 1) Código duplicado
  - 2) Variáveis com nomes não significativos
  - 3) Métodos adicionados erroneamente dentro de classes
  - 4) Código sem comentário

Refatoração

- ***Motivos para refatorar***:
  - 1 - Garantir que novas funcionalidades adicionadas ao programa estejam padronizadas
  - 2 - Melhorar a legibilidade do código
  - 3 - Encontrar e eliminar bugs que podem ainda não ter sido detectados
  - 4 - Programas mais rápido

Refatoração

- ***Técnicas de refatoração***:
  - 1) ***Extrair método:***
    - Um método pode estar realizando mais de uma função. Podemos separar as diferentes funções em novos métodos.

```java
public double calcularValorAluguel(int dias, double precoDiario) {
    double valor;

    if (dias > 7) {
        valor = dias * precoDiario * 0.9; // 10% desconto
    } else {
        valor = dias * precoDiario;
    }

    double imposto = valor * 0.05;
    valor = valor + imposto;

    return valor;
}
```

OBS: Exemplos de código criado com auxílios de IA

```java
public double calcularValorAluguel(int dias, double precoDiario) {
    double valorBase = calcularValorBase(dias, precoDiario);
    double valorComImposto = aplicarImposto(valorBase);
    return valorComImposto;
}

private double calcularValorBase(int dias, double precoDiario) {
    if (dias > 7) {
        return dias * precoDiario * 0.9;
    }
    return dias * precoDiario;
}

private double aplicarImposto(double valor) {
    return valor + (valor * 0.05);
}
```

Refatoração

- ***Técnicas de refatoração***:
  - 2) ***Mover métodos:***
    - Um método pode estar localizado em um classe inadequada. Métodos devem ser declarados em classes que fazem sentido aquele métodos estar lá.
      - Princípio da responsabilidade única

```java
public class Cliente {

    private String nome;
    private int pontosFidelidade;

    public Cliente(String nome) {
        this.nome = nome;
        this.pontosFidelidade = 0;
    }

    public double calcularValorAluguel(int dias, double precoDiario) {
        double valor = dias * precoDiario;

        if (dias > 7) {
            valor *= 0.9; // desconto
        }

        return valor;
    }
```

Refatoração

```java
public class Aluguel {

    private int dias;
    private double precoDiario;

    public Aluguel(int dias, double precoDiario) {
        this.dias = dias;
        this.precoDiario = precoDiario;
    }

    public double calcularValor() {
        double valor = dias * precoDiario;

        if (dias > 7) {
            valor *= 0.9;
        }

        return valor;
    }
```

```java
public class Cliente {

    private String nome;
    private int pontosFidelidade;

    public Cliente(String nome) {
        this.nome = nome;
        this.pontosFidelidade = 0;
    }
}
```

Refatoração

- ***Técnicas de refatoração***:
  - 3) ***Alteração usando polimorfismo:***
    - Pode ser útil para eliminar IF/ELSE aninhados dentro de um método. Os diferentes comportamentos pode ser transferidos um para cada subclasse nova.
      - A função do método é a mesma, porém com implementação diferente

```java
public class Aluguel {

    private int dias;
    private Filme filme;

    public Aluguel(Filme filme, int dias) {
        this.filme = filme;
        this.dias = dias;
    }

    public double calcularValor() {
        if (filme.getTipo().equals("NORMAL")) {
            return dias * 5.0;
        } else if (filme.getTipo().equals("LANCAMENTO")) {
            return dias * 8.0;
        } else if (filme.getTipo().equals("INFANTIL")) {
            return dias * 3.0;
        }
```

Refatoração

```
public abstract class Filme {

    protected double precoDiario;

    public Filme(double precoDiario) {
        this.precoDiario = precoDiario;
    }

    public abstract double calcularPreco(int dias);
}
```

Refatoração

```java
public class FilmeNormal extends Filme {

    public FilmeNormal() {
        super(5.0);
    }

    @Override
    public double calcularPreco(int dias) {
        return dias * precoDiario;
    }
}
```

Refatoração

```java
public class FilmeLancamento extends Filme {

    public FilmeLancamento() {
        super(8.0);
    }

    @Override
    public double calcularPreco(int dias) {
        return dias * precoDiario;
    }
}
```

Refatoração

```java
public class FilmeInfantil extends Filme {

    public FilmeInfantil() {
        super(3.0);
    }

    @Override
    public double calcularPreco(int dias) {
        return dias * precoDiario;
    }
}
```

Conteúdo da aula

- Introdução

- Arranjos
- Matrizes
- Classe Array
- Classe ArrayList

- **Conclusão**
- **Exercícios Propostos**
- **Referências bibliográficas**

Clean Code

- ***Definição***: Um código limpo de “sujeiras”, inconsistências e bem escrito pode gerar **benefícios** a empresa e aos desenvolvedores que irão gerenciar ele:

  - Manutenibilidade
  - Legibilidade
  - Eficiência

Clean Code

Fonte: https://youtu.be/VuN0L5UGsbk

Conteúdo da aula

- Introdução

- Arranjos
- Matrizes
- Classe Array
- Classe ArrayList

- **Conclusão**
- **Exercícios Propostos**
- **Referências bibliográficas**

Teste Unitário

- ***Definição***: Testar classes e/ou métodos de maneira independente do programa

  - Para o teste unitários de programas java no VS Code serão necessários alguns recursos

    - A extensão JAVA Extension Pack (maven e junit)
    - JDK (versão 1.8 ou posterior)
    - Visual Studio Code (versão 1.59.0 ou posterior)

Teste Unitário

Fonte: https://www.youtube.com/watch?v=Bxfy6ysoOzU

Conteúdo da aula

- Introdução

- Arranjos
- Matrizes
- Classe Array
- Classe ArrayList

- **Conclusão**
- **Exercícios Propostos**
- **Referências bibliográficas**

Conclusão

-

Exercícios Propostos

Bibliografia Básica

B. MEYER, **Object-oriented software construction**, Prentice-Hall, 1997.

GAMMA, Erich; et al. **Padrões de projeto: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2008.

BARNES, David. J.; KÖLLING, Michael. **Programação orientada a objetos com Java**. 4a ed. São Paulo: Prentice Hall, 2009.

Bibliografia Complementar

FOWLER, M. Refatoração: **Aperfeiçoando o Projeto de Código Existente**. 1a Ed. Bookman, 2004.

FREEMAN, E.; FREEMAN, E. **Use a Cabeça! Padrões de Projetos (Design Patterns)**. 2a ed. Alta Books, 2007.

Material de apoio

https://www.devmedia.com.br/introducao-a-refatoracao/21377

https://www.locaweb.com.br/blog/temas/codigo-aberto/clean-code/

https://code.visualstudio.com/docs/java/java-testing